UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

Informatics Institute of Technology

University of Westminster

COURSE – (LEADING TO)	MSc APPLIED AI
MODULE NAME	FOUNDATIONS OF ARTIFICIAL INTELLIGENCE
MODULE ID	COSC013C
ASSIGNMENT ID	CW1 - REPORT
DESCRIPTION	RESEARCH, JUSTIFY AND IMPLEMENT ONE OR A COMBINATION OF AI TECHNIQUES TO ACHIEVE THE STATED GOAL.
SUBMISSION DATE	2026-08-05
SUBMISSION VIA	REPORT WILL BE SUBMITTED THROUGH BLACKBOARD. LINKS TO THE SUPPORTING MATERIALS ARE ATTACHED AT THE END OF THE REPORT AND WILL REDIRECT TO A GITHUB REPOSITORY.
MATERIALS INCLUDED	REPORT PROPOSAL NOTEBOOK DEMONSTRATION VIDEO

STUDENT NAME	: K.P. SADUSHA NADEESH DE SILVA
UOW ID	: 22238997
IIT ID	: 20251211

1. Introduction and problem statement

Navigation apps such as Google Maps, Waze and Apple Maps make it easy to travel from one place to another. However, they mainly focus on finding the shortest or fastest route. While this works well for simple journeys, it does not consider personal preferences or different travel situations.

In real life, people often need to make multiple stops during a trip, such as visiting an ATM, getting coffee or stopping at a pharmacy. Their choices are influenced by factors like weather, traffic, safety, budget and personal preferences. For example, a budget traveller may prefer the shortest route with affordable places, while a luxury traveller may be willing to travel further for a higher rated location. Traditional navigation systems usually ignore these factors and simply recommend the nearest places.

This project aims to solve this problem by developing a solution, an AI Travel assistant that creates more personalised travel routes. Instead of using only the shortest path, the system combines several AI techniques to recommend routes that better match the user's needs.

The project uses three main AI approaches:

- Graph Search Algorithms: A* Search are used to find efficient routes.
- Knowledge Representation: Rule based logic adjusts routes based on conditions such as weather, traffic and safety.
- Machine Learning: Decision Tree Regressor models predict user preferences and rank Point of Interest (POI) based on different travel profiles.

To plan journeys with multiple stops, the system also applies route optimisation based on the Travelling Salesperson Problem. It selects suitable POIs, determines the best visiting order and displays the final route on a folium map. The project is implemented in a Jupyter Notebook using OpenStreetMap data and open-source Python libraries.

2. Background and Literature Review

This project is based on key concepts of graph theory, knowledge representation, machine learning and route optimisation.

2.1 Graph Theory and Heuristic Search

Graph theory is the foundation of modern navigation systems, in this project a road network is represented as a graph, where nodes represent intersections or locations and edges represents the roads connecting them. The goal is to find the shortest or lowest cost path between a starting point and a destination.

The project includes a Dijkstra's algorithm with A* algorithm comparison, for better clarity and demonstration.

One of the most known path-finding algorithms is **Dijkstra's Algorithm**, introduced by Edsger in 1959, which guarantees the shortest path by exploring nodes based on the lowest travel cost, but it searches in many directions, it can be slow for large road networks.

To improve efficiency, the **A* Search Algorithm** was introduced in 1968 by Peter Hart, Nils Nilsson, and Bertram Raphael. Unlike Dijkstra's algorithm, A* uses a **heuristic** to estimate the remaining distance to the destination. It combines the distance already travelled with the estimated distance to the goal. This estimate never exceeds the actual road distance, A* can find the shortest route while exploring fewer nodes and resources than Dijkstra's algorithm. This makes it faster and more suitable for real world navigation.

2.2 Knowledge Representation and Context Awareness

Knowledge Representation is a branch of Artificial Intelligence that focuses on storing knowledge in a way that computers can understand and use. One common method is a **rule-based system**, where decisions are made using simple **IF THEN** rules.

In navigation, Knowledge Representation helps the solution consider the real-world conditions instead of only finding the shortest route. Factors such as weather, traffic, and time that can affect which route is the most suitable.

In this project, rule-based logic is used to make the routing system more context-aware. For example, road routes can be assigned higher travel costs during rush hour, routes through less safe areas that can be avoided at night, and outdoor POIs can be given lower priority during bad weather. These rules are applied before the route is calculated, allowing the search algorithm to choose a more precise route for the user.

By combining rule-based reasoning with graph search algorithms, the solution can produce routes that are both efficient and better suited to the user's travel conditions.

2.3 Machine Learning and Supervised Personalization

Rule-based systems work well for most of the general scenarios, but they cannot easily represent personal preferences, different user’s opinions like distance, ratings, or travel time differently when choosing a café or other point of interest between destination.

So, the project uses **Machine Learning (ML)**, specifically **supervised learning**. The model is trained using sample data which is synthetic that includes information about different venues and user preference scores. From this data, model learns the patterns that influence a user's choices and can recommend places for different travel profiles.

This project uses a **Decision Tree Regressor**, which predicts scores by learning a series of decision rules from the training data. Decision Trees were selected because they are accurate which can handle different types of data without requiring complex data preparation. This allows

the solution to provide more personalised recommendations instead of relying on fixed scoring rules.

2.4 The Travelling Salesperson Problem (TSP)

When the route has multiple stops, the system faces a more complex problem than finding a simple path from one location to another. This problem is related to the **Travelling Salesperson Problem (TSP)**, where the goal is to find the shortest route that visits a set of locations in the best possible order avoiding unnecessary detours and distance.

Basic approach would require checking every possible order of stops, which becomes too slow when many stops are considered, will becomes difficult

Project focuses on a small urban travel scenario with around 3 to 4 stops which also practical to test all possible stop orders and find the best route.

Instead of calculating a new A* path for every possible route combination, the solution first creates a **distance matrix** and stores the shortest travel distance between each pair of locations to quickly compare different stop sequences using these pre calculated distances.

This approach combines graph search and route optimization, to generate the best order for visiting multiple POIs.

3. System Architecture and Methodology

The AI Travel Assistant uses a step-by-step modular design instead of a single routing process. Each stage adds a different type of intelligence, combining route search, rule-based decisions, and machine learning to create a personalized travel plan.

The system follows these main steps:

1. **User Input and Location Processing:**
   The system takes inputs such as starting point, destination, required POI categories, user profile (Budget or Luxury), weather conditions, and time of day.

2. **Map Data Collection:**
   The required road network data is downloaded dynamically from OpenStreetMap based on the selected area instead of using a large, fixed database.

3. **Applying Context Rules:**
   A rule-based system adjusts road costs based on conditions such as traffic, weather, and safety before route calculation begins.

4. **Finding the Base Route:**
   The A* search algorithm is used to calculate the most efficient route between the start and destination without additional stops. This acts as the baseline route.

5. **Finding Suitable POIs:**
   The system searches for relevant POIs near the baseline route based on the categories requested by the user.

6. **Personalised Recommendations:**
   The selected POIs are evaluated using Decision Tree Regressor models. The model ranks locations based on the user's travel profile and selects the most suitable options.

7. **Optimising Stop Order:**
   The selected POIs are treated as stops in the journey. A distance matrix is created, and the best visiting order is calculated using TSP optimisation.

8. **Creating the Final Route:**
   The A* algorithm is used again between each selected stop to generate the complete personalised travel route.

This approach allows the system to combine efficient navigation with user preferences and real-world conditions.

4. Data Collection and Preprocessing

Quality data is essential for any AI system. In this project, geospatial data is used instead of simple datasets because it includes both road locations and the connections between them. This information allows the routing algorithms to generate accurate and efficient routes and train model.

4.1 Dynamic OpenStreetMap

To keep the application lightweight, the system does not store large road network datasets. Instead, it uses the **OSMnx** Python library to download road network data from **OpenStreetMap (OSM)** whenever a user selects a location.

For example, if a user requests a route in Colombo or Kandy, the system downloads only the road network for that area. This reduces storage requirements and allows the application to work in different locations.

To improve performance, the downloaded road network is saved locally as a **graphml** file. This means the data can be reused later without downloading it again, reducing loading time and unnecessary API requests.

The road network is represented as a graph made up of:

- **Nodes:** Locations such as road intersections, each with a unique ID, latitude, and longitude.
- **Edges:** Road segments connecting the nodes, containing information such as road length, road type, speed limit, and estimated travel time.

This graph provides the information needed for the routing algorithms to calculate efficient travel routes.

4.2 Spatial Projection and the EPSG Ecosystem

To calculate distances accurately, the system cannot rely directly on latitude and longitude coordinates because they are measured in degrees rather than meters. Using them for distance calculations can produce inaccurate results.

To solve this, the project uses the **GeoPandas** library to convert the data from **EPSG:4326 (WGS84)** to **EPSG:3857**, a coordinate system that measures distances in metres. This makes it possible to calculate distances and create accurate spatial buffers.

For example, the system can search for all Points of Interest (POIs) within **500 metres** of the route. After these calculations are completed, the data is converted back to **EPSG:4326** so it can be displayed correctly on the interactive map.

Using the correct coordinate system helps ensure that nearby POIs are identified accurately and that the route calculations are reliable.

4.3 Synthetic Data Generation for Supervised Learning

In a real-world application, the recommendation system would be trained using large amounts of user data, such as visited locations, route choices, ratings, and feedback. However, since this academic project does not have access to real user data, synthetic data is created to simulate user preferences.

The system generates **10,000 sample records** for each user profile. Each sample includes four main features:

- **Detour score:** Measures the extra travel time needed to visit a POI.
- **Route distance score:** Represents how close the POI is to the planned route.
- **Category match score:** Shows how well the POI matches the user's requested category.
- **Review score:** A simulated rating value representing user feedback.

To create training labels for the Machine Learning model, the system uses custom scoring rules to represent different user behaviours.

- **Budget Traveller Profile:** Gives more importance to shorter distances and smaller detours, while giving less importance to ratings. This represents users who prefer convenient and affordable options.
- **Luxury Traveller Profile:** Gives more importance to high ratings and suitable venues, while being less concerned about extra travel distance. This represents users who are willing to travel further for better experiences.

Using synthetic data allows the system to test whether the Decision Tree Regressor can learn different user preferences and make personalised recommendations. It also provides a controlled environment for evaluating the ML model.

5. Algorithmic Implementation

The implementation phase involved converting the concepts of Graph Theory and Machine Learning into a working Python-based system. The aim was to transform the proposed algorithms and models into an efficient application that could generate personalised travel routes.

5.1 Custom Heuristic Search (Dijkstra and A*)

Although libraries like NetworkX already provide shortest path algorithms, this project implements **Dijkstra's Algorithm** and **A* Search** manually to better understand how these algorithms work.

The system uses Python's **heapq** module to manage the list of nodes that need to be checked. Dijkstra's Algorithm selects the next node based only on the distance travelled so far. A* improves this by also considering the estimated distance to the destination, allowing it to search more efficiently.

A*, the system uses the **Haversine formula** to estimate the distance between two locations. This helps the algorithm focus on nodes that are closer to the destination. The heuristic is designed to avoid overestimating the real travel cost, which ensures that A* can still find the best possible route while exploring fewer nodes.

5.2 The knowledge Representation (KR)

The Knowledge Representation component is implemented as a rule-based system that adjusts the routing process based on real-world conditions. Before running the routing algorithm, the system modifies road costs and POI scores according to the selected context.

The rules consider three main factors:

- **Time of Day:** During rush hour, roads classified as primary and secondary roads are given higher travel times to represent increased traffic levels.

- **Safety Conditions:** When night mode is enabled, smaller roads such as residential streets and paths receive higher costs. This encourages the algorithm to select safer routes where possible.
- **Weather Conditions:** Weather rules are applied during POI selection. When it is raining, outdoor locations such as tourist attractions receive lower preference scores, while indoor locations such as cafés and supermarkets become more favorable.

These rules allow the routing system to consider real-world conditions instead of focusing only on the shortest distance, resulting in more practical and user-friendly routes.

5.3 Optimizing the TSP with Distance Matrices

After the Machine Learning model selects the best POIs, the system needs to determine the most efficient order to visit them. A simple approach would be to run the A* algorithm for every possible stop order, but this would become slow because many different combinations need to be tested.

To improve performance, the system uses a **Distance Matrix**. Instead of calculating the route between locations repeatedly, it first calculates the travel time between all required points, including the start location, selected POIs, and destination.

The calculated distances are stored in a lookup table, allowing the system to quickly compare different stop sequences without running A* multiple times. This significantly reduces processing time and makes it practical to solve small-scale multi-stop routing problems.

This allows the solution to efficiently find the best visiting order while maintaining accurate travel cost calculations.

6. Performance Evaluation and Critical Analysis

To test whether the AI system works as expected, the project evaluates and tests different parts of the project, including search performance, machine learning accuracy, and the quality of the final routes. The evaluation compares the results of each component to measure how effectively the system improves route planning and personalisation.

6.1 Quantitative Search Efficiency of Dijkstra with A*

The first evaluation focused on comparing the performance of Dijkstra's Algorithm and A* Search. Both algorithms were tested by finding the shortest route between two locations in Colombo, Sri Lanka, covering 4.76 **km**.

The results showed that both algorithms produced the same optimal route, confirming that the Haversine heuristic used in A* did not affect the accuracy of the result.

However, the main difference was in search efficiency. Dijkstra's Algorithm explored **1,809 nodes** before reaching the destination, while A* explored only **87 nodes**.

This means A* reduced the number of explored nodes by approximately **95%**. By using the estimated distance to the destination, A* was able to focus its search on more relevant areas instead of exploring the graph in all directions. This improvement is important because the system repeatedly uses route calculations during multi stop optimisation, where faster searches help maintain overall performance.

6.2 Machine Learning Evaluation

The Machine Learning component was evaluated to check whether the Decision Tree Regressor models could learn user preferences from the training data and make accurate predictions.

The models were tested using a **20% holdout test set**, containing **2,000 synthetic samples** that were not used during training. The results showed strong performance for both user profiles:

- **Budget Model:** Achieved an MSE of **0.0038** and an R^2 score of **0.8695**.
- **Luxury Model:** Achieved an MSE of **0.0011** and an R^2 score of **0.9758**.

The high R^2 scores show that both models successfully learned the patterns in the synthetic preference data.

The feature importance also showed that the models learned the expected behaviours:

- **Budget Model:** Focused mainly on detour (**61%**) and distance (**35%**), while giving no importance to reviews. This matches the behaviour of users who prefer shorter and more convenient routes.
- **Luxury Model:** Focused mainly on reviews (**57%**) and category matching (**43%**), while ignoring distance and detour. This matches users who are willing to travel further for higher-quality locations.

These results confirm that the models were able to learn different travel preferences and generate personalised recommendations.

6.3 Scenario Analysis

The complete system was tested using different travel scenarios to check whether the Machine Learning models could create different routes based on user preferences.

In a test involving a café and an ATM, the **Budget profile** selected locations close to the main route, adding less than **0.2 km** of extra travel, even when the locations had lower ratings (for example, **3.1/5.0**). In comparison, the **Luxury profile** selected higher-rated locations and accepted a larger detour of around **1.0 km** to visit a café with a **4.9/5.0** rating.

This difference shows that the Machine Learning model successfully changes route recommendations based on user preferences instead of using the same route for everyone.

The system was also tested in multiple cities to check whether it could work beyond one location. After testing with both **Colombo and Kandy, Sri Lanka**, the system successfully downloaded road data, found suitable POIs, ranked them using the ML models, and generated an optimised route. This shows that the system is flexible and does not depend on fixed data from a single city.

The ML approach was also compared with a simple distance-based method. A distance-only system would always recommend the closest locations, meaning Budget and Luxury users would receive similar results. The ML model improves this by considering different user preferences. For example, Luxury users are prioritised toward highly rated locations, while Budget users are guided toward shorter and more convenient options.

This confirms that the Machine Learning component adds real value by creating more personalised travel recommendations.

6.4 Automation Testing and Validation

To check that the system works correctly and consistently, the notebook includes automated tests that run during each execution. These tests verify the main functions of the routing pipeline.

The tests check that:

- Both Dijkstra's Algorithm and A* Search successfully generate valid routes.
- All calculated distances and travel times are positive values.
- A* explores fewer nodes than Dijkstra for the same route.
- At least one suitable POI is found for each requested category.
- The final route with additional stops is not shorter than the direct route, confirming that POI detours are included correctly.

All tests passed successfully for both **Colombo and Kandy** scenarios. This confirms that the system can produce valid results across different locations and that the main components of the pipeline are working as expected.

7. Ethical Consideration and Responsible AI

The use of Artificial Intelligence in navigation and recommendation systems requires careful consideration of ethical issues. Since these systems can influence user decisions, it is important to ensure that they are developed and used responsibly.

The project considers factors such as data privacy, transparency, and fairness to ensure that the AI system provides useful recommendations without causing unnecessary risks or misuse of user information.

7.1 Algorithmic Bias and Synthetic Data

One limitation of this project is the use of synthetic data for training the Machine Learning models. The Budget and Luxury user profiles are created based on assumptions made by the developer, which may not fully represent real user behaviour.

For example, assuming that Budget users only care about distance and Luxury users only care about ratings can be too simple. Real users may consider many other factors, such as safety, convenience, or personal preferences.

In a real-world system, the models should be trained and updated using diverse user data. This would help reduce bias and allow the system to better understand different types of travellers.

7.2 Generative AI Usage

Google Gemini, Anthropic Claude and OpenAi Codex models (2025–2026) were used during development for limited support tasks, including brainstorming algorithmic edge cases, debugging visualisation issues, and assisting with synthetic data generation ideas. All generated suggestions were reviewed, tested, and modified by the author before implementation. Mainly used for:

- **Algorithm Planning:** Exploring possible edge cases and improving understanding of routing algorithms.
- **Debugging Support:** Helping identify issues with code syntax and map visualisation settings.
- **Synthetic Data Preparation:** Assisting with creating sample data distributions for testing the Machine Learning models.

All AI assisted suggestions and code examples were reviewed, tested, and modified by the developer. The main implementation, algorithm design, testing, and evaluation were completed independently to ensure full understanding and ownership of the final system.

8. Limitations and Future Enhancements

Although the AI Travel Assistant successfully combines different AI techniques, the current system has some limitations that could be improved in future work.

Scalability of Route Optimisation:
The current system uses an exact method to test all possible stop orders for the TSP problem.

This works well for small trips with around 3 to 4 stops, but it becomes slow when the number of stops increases. Future versions could use optimisation methods such as Genetic Algorithms or Simulated Annealing to handle larger numbers of locations more efficiently.

Limited Real-Time Data:
The system currently uses static road information from OpenStreetMap, including estimated speeds and travel times. Although the rule system adjusts routes for conditions such as rush hour, it cannot fully represent real traffic situations. A future system could include live traffic data to provide more accurate travel times.

Heuristic Accuracy in Complex Areas:
The A* algorithm uses the Haversine distance as an estimate between locations. While this works well, it assumes a straight-line connection between points, which may not always match real road layouts. Areas with one-way streets, rivers, or complex road networks may require more advanced routing techniques to improve efficiency.

These improvements would make the system more scalable, accurate, and suitable for real-world navigation applications.

Conclusion

This project successfully developed a hybrid AI Travel Assistant that goes beyond traditional shortest-path navigation. Instead of only finding the fastest route, the system combines multiple AI techniques to create more personalised and context-aware travel plans.

The project combines several AI approaches: **Graph Theory** using Dijkstra and A* algorithms for route finding, **Knowledge Representation** for handling conditions such as weather and time of day, **Machine Learning** for understanding user preferences, and **Combinatorial Optimisation** for arranging multiple stops efficiently.

The evaluation results show that the system performs effectively. A* reduced the number of explored nodes by **95%** compared to Dijkstra, while the Machine Learning models achieved strong accuracy in predicting user preferences (**$R^2 > 0.97$**).

Overall, this project demonstrates that modern navigation systems can be improved by considering not only the shortest route, but also the user's preferences and real-world travel conditions. The hybrid AI approach provides a strong foundation for building more intelligent and personalised travel assistants.

10. References

Boeing, G. (2017) 'OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks', ***Computers, Environment and Urban Systems*****, 65, pp. 126–139. Available at: https://doi.org/10.1016/j.compenvurbsys.2017.05.004**

Breiman, L., Friedman, J., Stone, C. J. and Olshen, R. A. (1984) ***Classification and Regression Trees*****. Boca Raton: CRC Press.**

Dijkstra, E. W. (1959) 'A note on two problems in connexion with graphs', ***Numerische Mathematik*****, 1(1), pp. 269–271. Available at: https://doi.org/10.1007/BF01386390**

Hart, P. E., Nilsson, N. J. and Raphael, B. (1968) 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths', ***IEEE Transactions on Systems Science and Cybernetics*****, 4(2), pp. 100–107. Available at: https://doi.org/10.1109/TSSC.1968.300136**

Held, M. and Karp, R. M. (1962) 'A dynamic programming approach to sequencing problems', ***Journal of the Society for Industrial and Applied Mathematics*****, 10(1), pp. 196–210.**

Jordahl, K. et al. (2020) ***GeoPandas: Python tools for geographic data*****. Available at: https://github.com/geopandas/geopandas**

OpenStreetMap contributors (2026) ***Planet dump*****. Available at: https://planet.osm.org (Accessed: July 2026). Licensed under the Open Data Commons Open Database License (ODbL).**

Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', ***Journal of Machine Learning Research*****, 12, pp. 2825–2830.**

Russell, S. and Norvig, P. (2020) ***Artificial Intelligence: A Modern Approach*****. 4th edn. Harlow: Pearson.**